

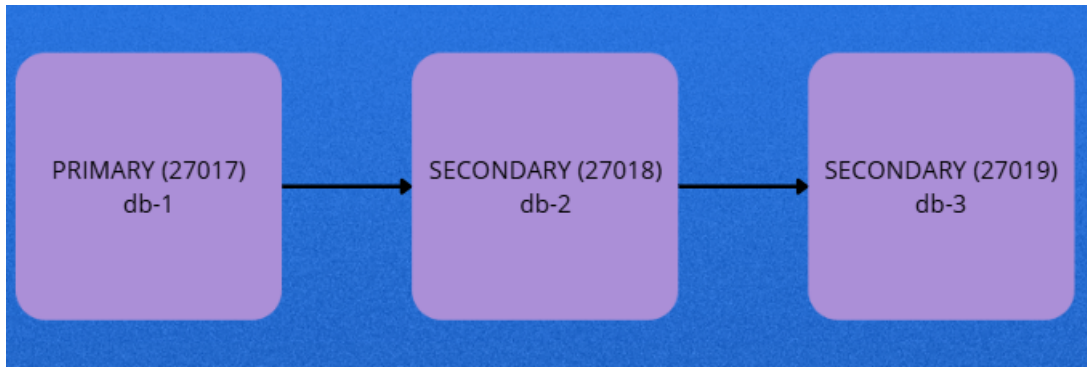
Bases de Données Avancées
Interface web Django
RAPPORT Final

Felipe Mateus Falcão Barreto

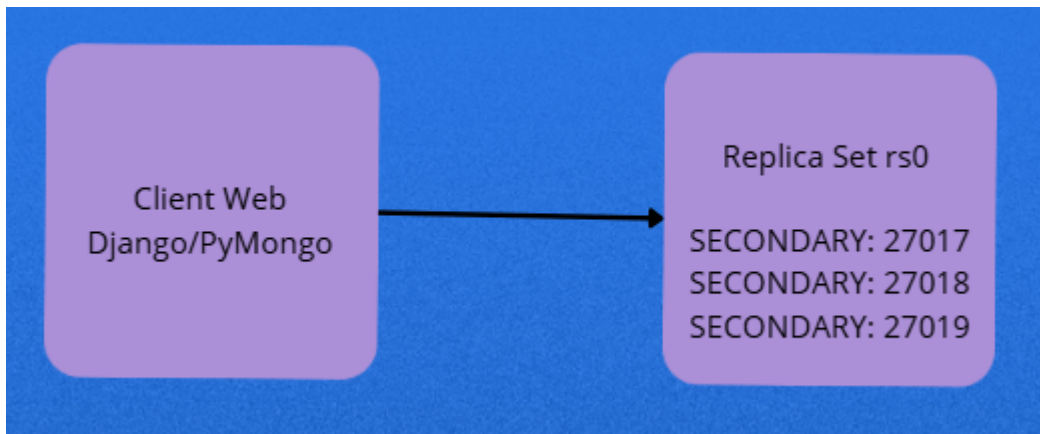
1. Introduction

Le projet CineExplorer consiste à développer une application web d'interrogation et d'analyse de données cinématographiques, intégrant SQLite pour les données relationnelles et MongoDB pour les données documentaires pré-agrégées. L'objectif principal est de garantir une haute disponibilité, des performances optimales et une interface utilisateur intuitive, tout en explorant les fonctionnalités de réplication et de basculement offertes par MongoDB.

2. Architecture globale (schéma)



MongoDB a été configuré dans un ensemble de répliques avec trois nœuds localhost sur 27017, 27018 et 27019 pour assurer la tolérance aux pannes, avec un nœud PRIMARY (écriture) et deux nœuds SECONDARY (copie de données).



- SQLite funciona localmente para consultas relacionais rápidas (listas de filmes, filtros).
- MongoDB armazena dados documentais complexos, agregados e historicamente consistentes.

MongoDB a été configuré dans un ensemble de répliques avec trois nœuds localhost sur 27017, 27018 et 27019 pour assurer la tolérance aux pannes, avec un nœud PRIMARY (écriture) et deux nœuds SECONDARY (copie de données).

2.1 Configuration du replica set

Script setup_replica.sh:

- Arrête les processus mongod en cours.
- Démarre trois instances MongoDB sur les ports 27017, 27018 et 27019 avec des répertoires de données distincts.
- Initialise le jeu de réplicas rs0 via mongosh.

Script test_failover.py:

- Teste la lecture, l'écriture, le failover et la resynchronisation des nœuds.
- Mesure le temps de failover et le comportement en cas de panne simple et double

2.2 Configuration de Django

- Connexion à SQLite via settings.py.
- Connexion à MongoDB via PyMongo, pointant vers le jeu de réplicas contenant tous les nœuds.
- Modèles HTML de base (home.html, stats.html) intégrés aux vues Django.



Welcome to CineExplorer!

Total movies in database: 110577



Database Statistics

Total movies in SQLite: 110577

Total movies in MongoDB: 110577

3. Choix technologiques justifiés

- Django: un framework robuste pour les applications web offrant une intégration performante.
- SQLite: rapide et efficace pour les requêtes relationnelles simples.
- MongoDB: adapté aux documents pré-groupés et aux requêtes d'agrégation complexes.
- Bootstrap/Tailwind CSS: pour une expérience utilisateur moderne et réactive.
- Chart.js: pour la visualisation de statistiques.

4. Description des fonctionnalités (non mis en œuvre)

Page d'accueil (/)

- Statistiques : nombre de films, acteurs, réalisateurs
- Top 10 des films les mieux notés
- Formulaire de recherche rapide
- Films récemment ajoutés ou aléatoires

Liste des films (/movies/)

- Pagination (20 films par page)
- Filtres : genre, année (range), note minimale
- Tri : titre, année, note (ASC/DESC)
- Affichage en grille ou liste

Détail d'un film (/movies/<id>/)

- Toutes les informations (depuis MongoDB)
- Casting complet avec personnages
- Réalisateurs et scénaristes
- Titres alternatifs par région
- Films similaires (même genre/réalisateur)

Recherche (/search/?q=...)

- Recherche par titre de film
- Recherche par nom de personne
- Résultats groupés par type

Statistiques (/status/)

- Films par genre (bar chart)
- Films par décennie (line chart)
- Distribution des notes (histogram)
- Top 10 acteurs prolifiques

5. Stratégie multi-bases

- SQLite pour les données structurées et les requêtes rapides
- MongoDB pour les données documentaires complexes, les agrégations et l'historique des données
- Intégration via PyMongo et Django

6. Benchmarks de performance

Écriture/Insertion: insertion de documents dans MongoDB via un ensemble de répliquas.

Réplication: vérification des données sur le nœud SECONDARY après leur insertion dans le nœud PRIMARY.

Failover: interruption du nœud PRIMARY, observation de l'élection et resynchronisation.

Double panne: deux nœuds indisponibles, vérification de la perte du quorum.

Le temps de failover observé était d'environ 30 secondes. Les lectures restent possibles sur le nœud SECONDARY, les écritures sont bloquées pendant l'élection. La réinitialisation d'un nœud interrompu permet une resynchronisation automatique en tant que nœud SECONDARY. Une double panne bloque les opérations d'écriture, préservant ainsi la cohérence des données.

7. Difficultés et solutions

- **SQL vs NoSQL:** Découvrir les différences et choisir le type de données à utiliser dans chaque base de données.
Solution: SQLite pour les données relationnelles, MongoDB pour les documents agrégés.
- **Intégration avec Django:** réconciliation de deux bases de données distinctes et structuration de l'interface.
Solution: Une approche basée sur des couches de services séparées et des modèles simplifiés a été adoptée.
- **Tests de failover:** Simuler des pannes et mesurer la tolérance aux pannes.
Solution: Scripts automatisés pour la réplication, la lecture et l'écriture.

8. Accès au code source

L'intégralité du code source du projet est disponible dans le dépôt GitHub:

<https://github.com/felipemfb/Cineexplorer>